

Child Vocabulary Acquisition Simulation – Codebase Description

Developer Reference

Introduction.....	2	Utterances.java.....	9
Project Structure.....	2	Data structures.....	9
Architecture Overview.....	2	Static initialisation.....	9
JaCaMo Framework.....	2	Observable properties.....	9
System Architecture Diagram.....	3	Operations.....	9
Build System and Dependencies.....	3	Synchroniser.java.....	9
Gradle Configuration.....	3	State machine.....	10
Dependencies.....	3	Static state.....	10
Build Tasks.....	3	Observable properties.....	10
Configuration System.....	3	Operations.....	10
simulation.conf.....	3	Private methods.....	11
ConfigFile.java.....	4	DataLogger.java.....	11
Supporting Classes.....	4	Static state.....	11
Custom Launcher.....	4	Operations.....	11
VocabLauncher.java.....	4	Static methods.....	11
Startup sequence within createAgs():.....	5	WordsLearnt.java.....	11
Key design note:.....	5	Data structure.....	11
Agent Layer (Jason / AgentSpeak).....	5	Operations.....	11
Common Includes.....	5	Static methods.....	11
common.asl.....	5	Organisational Layer (Moise).....	12
child_common.asl.....	5	learning_envron.xml.....	12
adult_common.asl.....	6	Structural specification.....	12
Child Agent.....	6	Functional specification.....	12
Word data structure.....	6	Normative specification.....	12
Plans.....	6	JaCaMo Project File.....	12
Parent Agent.....	7	Simulation Lifecycle.....	13
Plans.....	7	Startup Sequence.....	13
Teacher Agent.....	7	Daily Cycle.....	13
Initial belief.....	7	Yearly Progression.....	13
Plans.....	7	Shutdown Sequence.....	14
Environment Layer (CArtAgO Artefacts).....	8	Word Learning Algorithm.....	14
Library.java.....	8	Concurrency and Synchronisation.....	14
Data structures.....	8	Data Flow.....	15
Static initialisation.....	8	Logging Configuration.....	16
Observable properties.....	8	Known Limitations and Unimplemented Features.....	16
Operations.....	8		

Introduction

This document provides a detailed, developer-focused description of the Child Vocabulary Acquisition (CVA) simulation codebase. It is intended for developers who wish to understand, maintain, or extend the system.

The CVA simulation models how children acquire vocabulary through exposure to spoken language (at home) and written text (at school), with exposure volumes determined by socio-economic status (SES). It is built on the JaCaMo multi-agent platform, version 1.3.0.

Project Structure

```
child_vocabulary_acquisition/
+-- build.gradle           # Gradle build configuration
+-- settings.gradle       # Gradle settings
+-- child_vocabulary_acquisition.jcm # JaCaMo project definition
+-- simulation.conf       # Runtime configuration (TOML)
+-- logging.properties   # Java logging configuration
+-- README.md            # Project readme
+-- gradlew / gradlew.bat # Gradle wrapper scripts
+-- src/
|   +-- agt/             # Agent source files (AgentSpeak)
|   |   +-- child.asl    # Child agent plans
|   |   +-- parent.asl   # Parent agent plans
|   |   +-- teacher.asl  # Teacher agent plans
|   |   +-- inc/         # Shared agent includes
|   |   |   +-- common.asl # Plans common to all agents
|   |   |   +-- child_common.asl # Plans common to child agents
|   |   |   +-- adult_common.asl # Plans common to adult agents
|   +-- env/            # Java environment classes
|   |   +-- VocabLauncher.java # Custom JaCaMo launcher
|   |   +-- Synchroniser.java # Simulation coordination artefact
|   |   +-- Library.java     # Book corpus artefact
|   |   +-- Utterances.java   # Speech corpus artefact
|   |   +-- DataLogger.java   # Statistics recording artefact
|   |   +-- WordsLearnt.java   # Word-level tracking artefact
|   |   +-- ConfigFile.java    # TOML configuration parser
|   |   +-- Agents.java       # Agent configuration data holder
|   |   +-- Environment.java   # Environment path configuration
|   |   +-- SES.java          # Socio-economic status model
|   |   +-- CircularQueue.java # Generic circular queue utility
|   |   +-- WordCountRow.java  # Annual statistics record
|   |   +-- WordLearntRow.java # Learned word record
|   |   +-- HashMapBB.java     # Custom indexed belief base
|   +-- int/                # Interaction definitions (empty)
|   +-- org/                # Moise organisation definitions
|       +-- learning_envIRON.xml # Organisational specification
```

Architecture Overview

JaCaMo Framework

JaCaMo integrates three technologies:

- **Jason** – A BDI (Belief-Desire-Intention) agent programming language based on AgentSpeak(L). Agents are defined in .asl files containing beliefs, goals, and plans.
- **CARTaGO** – Common ARTifact infrastructure for AGents in Open environments. Provides the artefact abstraction for environment entities that agents interact with via operations and observable properties.
- **Moise** – A model for organisational structures. Defines roles, groups, schemes, and norms that constrain agent behaviour.

System Architecture Diagram

The system comprises four workspaces:

Workspace	Artefacts	Purpose
classroom	Library	School environment – book reading
home_environ	Utterances	Home environment – speech exposure
sync	Synchroniser, DataLogger, WordsLearnt	Coordination and data recording
learning_environ	(Moise groups and schemes)	Organisational structure

Agent-artefact interactions:

- **Child agents** focus on: Library, Synchroniser, DataLogger, WordsLearnt, plus Moise group artefacts.
- **Parent agents** focus on: Utterances, Synchroniser, plus Moise group artefacts.
- **Teacher agent** focuses on: Library, Synchroniser, plus Moise group artefacts.

Build System and Dependencies

Gradle Configuration

The project uses Gradle with the Shadow plugin for fat JAR creation. Key configuration in `build.gradle`:

- **Entry point:** `vocab.VocabLauncher` (overrides the default `jacamo.infra.JaCaMoLauncher`)
- **Source sets:** `src/env`, `src/agt`, `src/org`, `src/int`, `src/java`
- **Default task:** `run`

Dependencies

Dependency	Version	Purpose
<code>org.jacamo:jacamo</code>	1.3.0	Multi-agent platform
<code>com.opencsv:opencsv</code>	5.12.0	CSV file parsing and writing
<code>com.electronwill.night-config:toml</code>	3.8.3	TOML configuration file parsing
<code>com.gradleup.shadow</code>	8.3.3	Fat JAR packaging (Gradle plugin)

Build Tasks

Task	Description
<code>gradle / gradle run</code>	Compile and run the simulation
<code>gradle shadowJar</code>	Create a self-contained executable JAR
<code>gradle clean</code>	Remove <code>bin/</code> , <code>build/</code> , and <code>log/</code> directories

Configuration System

`simulation.conf`

The runtime configuration file uses TOML syntax and contains three sections:

[agents] – Controls agent population:

Key	Type	Default	Range	Description
<code>child</code>	int	10	User discretion	Number of child agents
<code>max_siblings</code>	int	0	0–2	Siblings per child (not implemented)
<code>percent_single_parents</code>	float	100.0	0.0–100.0	Percentage of single-parent households
<code>age_start</code>	int	3	0–5	Starting age of children

Key	Type	Default	Range	Description
age_finish	int	8	5-10	Finishing age of children

`[[ses]]` – SES categories (array of tables). Each entry requires name (string) and qty (int) keys. Defaults: Welfare (8,767), Working (17,514), Professional (30,142).

`[environment.locations]` – Paths to data directories:

Key	Default	Description
utterances	<code>./resources/utterances</code>	Directory containing utterance CSV files
books	<code>./resources/books</code>	Directory containing book text files

ConfigFile.java

Package: vocab

Location: `src/env/ConfigFile.java`

Parses `simulation.conf` using the NightConfig TOML library. Key design decisions:

- All settings have compile-time defaults (defined as `static final` constants).
- Out-of-range numeric values are clamped rather than rejected.
- If `age_start == age_finish`, `age_finish` is incremented by 1.
- If no valid `[[ses]]` entries exist, the three defaults are used.
- Missing or blank file paths fall back to default resource locations.

Public API:

- `static ConfigFile createDefault()` – Returns a `ConfigFile` with all defaults.
- `static ConfigFile parse(Path configFile)` – Parses the given TOML file.
- `Agents getAgents() / HashSet<SES> getSES()` – Accessors.

Supporting Classes

Agents.java (`src/env/Agents.java`) Simple data holder with getters for child count, max siblings, percent single parents, age start, and age finish. Constructed by `ConfigFile`.

SES.java (`src/env/SES.java`) Represents a socio-economic status category with a name and daily word quantity. Equality is determined solely by name (via `equals/hashCode` override), allowing `HashSet` to deduplicate categories. Provides a `defaults()` factory method returning the three standard categories.

Environment.java (`src/env/Environment.java`) Holds two path strings: `utteranceLocation()` and `bookLocation()`.

CircularQueue.java (`src/env/CircularQueue.java`) Generic circular queue (`CircularQueue<T>`) used for round-robin distribution of SES categories across parent agents. Supports `add(Collection)` and `get()` (which advances and wraps the internal pointer).

Custom Launcher

VocabLauncher.java

Package: vocab

Location: `src/env/VocabLauncher.java`

Extends: `jacamo.infra.JaCaMoLauncher`

The custom launcher is the application entry point. It overrides `createAgs()` to inject dynamically generated agents before JaCaMo processes the `.jcm` file.

Startup sequence within `createAgs()`:

1. `loadCustomConfig()` – Locates and parses `simulation.conf` from the working directory.
2. `initialiseArtifacts()` – Calls static initialisers on `DataLogger`, `Synchroniser`, `Utterances`, and `Library` to pre-load data before the MAS starts.
3. `addChildAgents()` – Creates N child agents from a template `JaCaMoAgentParameters`:
 - Source file: `child.asl`
 - Belief base: `HashMapBB` with indexed pattern `word(key, _)` for O(1) word lookups
 - Artefact focuses: `classroom.library`, `sync.synchroniser`, `sync.datalogger`, `sync.wordsLearnt`
 - Role: child in the village group
 - Initial beliefs: `age(X)`, `activityState("Idle")`
 - Initial goals: `join_school`, `join_home_group(home_N)` (with randomised home assignment)
4. `addParentAgents()` – Creates parent agents:
 - Belief base: `HashMapBB` with indexed patterns `words_to_speak(_)` and `iterationsLeft(_)`
 - Total parents = $\text{numFamily} * (1 + (100 - \text{percentSingleParents}) / 100)$
 - First `numFamily` parents get `create_home_group(N)` goal and SES assignment via circular queue.
 - Extra parents (for two-parent households) get `join_home_group(home_N)` goal.
 - The SES queue starting position is randomised to avoid deterministic assignment.
5. `addSiblingAgents()` – Placeholder; not implemented.
6. `super.createAgs()` – Standard JaCaMo agent creation from the `.jcm` file (creates the teacher agent).

Key design note:

The `main()` method is a near-verbatim copy of `JaCaMoLauncher.main()`, changed only to instantiate `VocabLauncher` instead of `JaCaMoLauncher`. The `create()` method is also overridden to remove the `createController()` call. The `createCustomPlatforms()` method is also copied because it is private in the parent class.

Agent Layer (Jason / AgentSpeak)

All agents are written in `AgentSpeak(L)`, Jason's agent programming language. Agents operate on a BDI (Belief-Desire-Intention) cycle: they maintain beliefs about the world, adopt goals (desires), and select plans (intentions) to achieve those goals.

Common Includes

`common.asl`

File: `src/agt/inc/common.asl`

Included by all agents. Provides:

- Standard JaCaMo template includes (`common-cartago.asl`, `common-moise.asl`, `org-obedient.asl`).
- `print_listItems` – A recursive plan for printing list contents to the console (debugging utility).

`child_common.asl`

File: `src/agt/inc/child_common.asl`

Included by `child.asl`. Provides:

- `setState(State)`: Updates the `activityState` belief.
- `join_home_group(HomeGroup)`: Waits for the `Moise` group to exist, then focuses on it and adopts the offspring role.
- `join_school`: Waits for the `classroom` group, then focuses and adopts the student role.

- `+sync::newYear (atomic)`: At year-end, waits until idle, then retrieves aggregate statistics by querying the belief base (`words::unique_seen`, `words::unique_heard`, and `.count` of learned words) and reports them to `DataLogger` via `addAnnualStats`.
- `+sync::finalise (atomic)`: At simulation end, retrieves all learned words via `.findall(aoa(Word, Age), words::word(Word, [_,_ , Age]) & Age\==0, WordAoA)` and reports them to `WordsLearnt` via `addLearnedWords`, then signals `childAgentFinalised`.
- `+sync::agent_age(NewAge) (atomic)`: Updates the agent's age belief when the synchroniser increments the age.

adult_common.asl

File: `src/agt/inc/adult_common.asl`

Included by `parent.asl` and `teacher.asl`. Provides:

- `read_book_aloud_to_child(Title, Child)` – Retrieves book text from the Library artefact and sends it to the child via `listen_to_speech`.

Child Agent

File: `src/agt/child.asl`

Includes: `child_common.asl`

Word data structure

Each word is tracked as a belief in the words namespace: `words::word(Word,[Seen,Heard,AgeLearned])`, where:

- `Seen` – number of times the word has been encountered in books
- `Heard` – number of times the word has been encountered in speech
- `AgeLearned` – the age at which the word was "learnt" (0 if not yet learnt)

Word data is stored in the agent's belief base using `HashMapBB` – a custom belief base implementation (see *Concurrency and Synchronisation*) that provides $O(1)$ indexed lookups by word. The first time a word is queried, a test-goal plan in `child_common.asl` creates the initial belief with zeroed counters.

Plans

`word_heard_checker(Word)` – Queries the word's belief via `?words::word(Word, [Seen, Heard, AgeLearned])`. If this is the first time the word has been heard (`Heard == 0`), increments the `words::unique_heard` counter. Then updates the word belief with the incremented `Heard` count via `+words::word(Word, [Seen, Heard+1, AgeLearned])`.

`word_seen_checker(Word)` – Same as above but for the `Seen` count and `words::unique_seen` counter.

`try_learn_word(word)` – Queries the word's belief. If `AgeLearned == 0` and the word meets the learning threshold (see *Word Learning Algorithm*), updates the belief to set `AgeLearned` to the child's current age.

`read_a_book(Title)` **[source guard: must come from the teacher]** – Retrieves book text from the Library, iterates over each word calling `word_seen_checker` and `try_learn_word`, then notifies the teacher of completion via `finishedReading(Title)`.

`+listen_to_speech(words)` **[atomic]** – Triggered when another agent (typically the teacher) speaks words directly. Iterates over the word list, calls `word_heard_checker` and `try_learn_word` for each, then sends `finishedListening` to the speaker.

`+listen_to_utterances(Utterances, Counter)` **[atomic]** – Triggered by parent agents. Processes a batch of utterances (list of word lists). After processing, records a `finished(Counter, Parent)` belief. The corresponding `+finished(_, Parent)` plan waits until 20 batches have been received before sending

finishedUtterances back to the parent. This batching approach (20 iterations) prevents the agent's belief base from being overwhelmed by a single enormous message.

Parent Agent

File: src/agt/parent.asl

Includes: adult_common.asl

Plans

create_home_group(Number) – Creates a Moise sub-group (indv_home) under the village group:

1. Looks up and focuses on the village artefact.
2. Creates a new group named home_N.
3. Adopts the parent role.
4. Waits for the group to be well-formed.
5. Waits for all expected parents to join (via find_parents).
6. Distributes household information (parent list, SES) to all residents.
7. Signals the synchroniser via groupCreated.

join_home_group(GroupName) – For second parents in two-parent households. Waits for the group to exist, then focuses and adopts the parent role.

find_parents(NumParents) – Recursive plan that checks if the expected number of parents have joined. On failure, waits 100ms and retries.

+sync::status("StartHome") – Triggered by the synchroniser. Delegates to speak_to_child.

speak_to_child [**atomic**] – Core speech delivery plan:

1. Determines the total daily word count from SES configuration.
2. Divides by the number of parents in the household.
3. Delivers words in 20 iterations, each requesting $\text{DesiredWords} = \text{RemainingWords} / \text{Counter}$ words from the Utterances artefact via getBulkUtterances.
4. Sends each batch to the child via listen_to_utterances.

+finishedUtterances – Triggered when the child confirms all 20 batches processed. Removes the Signals finishedHome to the synchroniser.

Teacher Agent

File: src/agt/teacher.asl

Includes: adult_common.asl

The teacher agent is the only statically-defined agent (declared in the .jcm file rather than dynamically created).

Initial belief

students_found(0) – tracks how many students are in the class.

Plans

find_my_students – Waits for the synchroniser to reach “Ready” status, then:

1. Queries the classroom group for all agents playing the student role.
2. Records the student list and count.
3. Sends each student a belief identifying this agent as their teacher (school::my_teacher(Me)).

4. Signals `schoolReady` to the synchroniser.

`read_books` **[atomic]** – The main daily plan, triggered by `StartSchool`:

1. Signals `startSchool` to the synchroniser.
2. Checks the children's age and selects a strategy:
 - Age < 5: Always reads aloud (`read_book_aloud_to_child`).
 - Age 5–7: 50% chance of reading aloud vs. telling students to read independently.
 - Age > 7: Always tells students to read independently.

`tell_students_to_read_book` **[atomic]** – Selects a random book title, retrieves its word count, sends an achieve goal (`read_a_book(Title)`) to all students, and records `sent_instruction_to_read_book(Title, Student)` beliefs. Has a failure plan that silently succeeds if preconditions are not met.

`+finishedListening` **[atomic]** – Triggered when a child finishes listening to a read-aloud session. Tracks completion via `finished(Child)` beliefs. When all students have reported, cleans up beliefs and signals `finishSchool` to the synchroniser.

`+finishedReading(Title)` **[atomic]** – Triggered when a child finishes reading a book independently. Tracks completion by counting `finishedReading` beliefs. When all students have reported, cleans up `finishedReading` and `sent_instruction_to_read_book` beliefs and signals `finishSchool` to the synchroniser.

`+sync::status("StartSchool")` – Resets the `read_books` goal for each new day via `resetGoal`.

Environment Layer (CARTAgO Artefacts)

All artefacts extend `cartago.Artifact` and reside in the `vocab` package.

Library.java

Location: `src/env/Library.java`

Workspace: `classroom`

Manages the corpus of book texts.

Data structures

- `HashMap<String, String[]> books` – Maps filename to pre-split word arrays. Populated lazily on first access to each book.
- `ArrayList<String> bookTitles` – List of book filenames. Populated during static initialisation.
- `ReentrantLock loadingLock` – Ensures thread-safe lazy loading.

Static initialisation

`loadBookTitles(String path)` – Called by `VocabLauncher.initialiseArtifacts()`. Scans the specified directory for `.txt` files and stores their filenames in the `bookTitles` list.

Observable properties

- `book_count` – Total number of books available.

Operations

`get_bookTitleRandomly()` – Returns a random book filename.

`get_bookByTitle(String title)` – Returns the book’s text as a `String[]` (pre-split word array). If the book hasn’t been loaded yet, acquires `loadingLock` and reads the file from disk. Handles UTF-8 BOM (byte order mark: EF BB BF).

`getWordCountByBookTitle(String title)` – Returns the word count of a book (loads the book if necessary). This exists because Jason’s `.length` internal action causes a `StackOverflowError` on very long lists.

Utterances.java

Location: `src/env/Utterances.java`

Workspace: `home_environ`

Manages the corpus of child-directed speech utterances.

Data structures

- `HashMap<String, ArrayList<String[]>> utterancesMap` – Maps speaker codes to lists of pre-split utterance word arrays.
- `HashMap<String, ArrayList<String>> participantRoleToCodeMapping` – Maps participant roles to speaker code lists.

Static initialisation

`loadUtterances(String filePath)` – Called by `VocabLauncher`. Delegates to:

1. `loadUtterances()` – Processes CSV files containing utterances in their filename, extracting `speaker_code` and `gloss` columns. Utterances are pre-split into `String[]` word arrays during loading to avoid repeated splitting at access time.
2. `loadParticipants()` – Processes CSV files containing participants in their filename, extracting `role` and `id` columns.

Both use the private helper `HashMapFromCSVFiles()` which:

- Validates the base path exists and is a directory.
- Filters files by `.csv` extension and filename substring.
- Uses `OpenCSV`’s `CSVReaderHeaderAware` for header-aware parsing.
- Assumes the first requested column is the key, the second is the value.

Observable properties

- `num_speaker_codes_available` – Number of distinct speaker codes.
- `utterances_available` – Total number of utterances loaded.

Operations

`getRandomUtterance()` – Returns a random speaker code, a random utterance from that speaker (as a word array), and the utterance length.

`getBulkUtterances(double numWordsRequired)` – Accumulates random utterances until the total word count meets or exceeds `numWordsRequired`. Returns the collected utterances as an `Object[]` of `String[]` arrays and the actual word count provided.

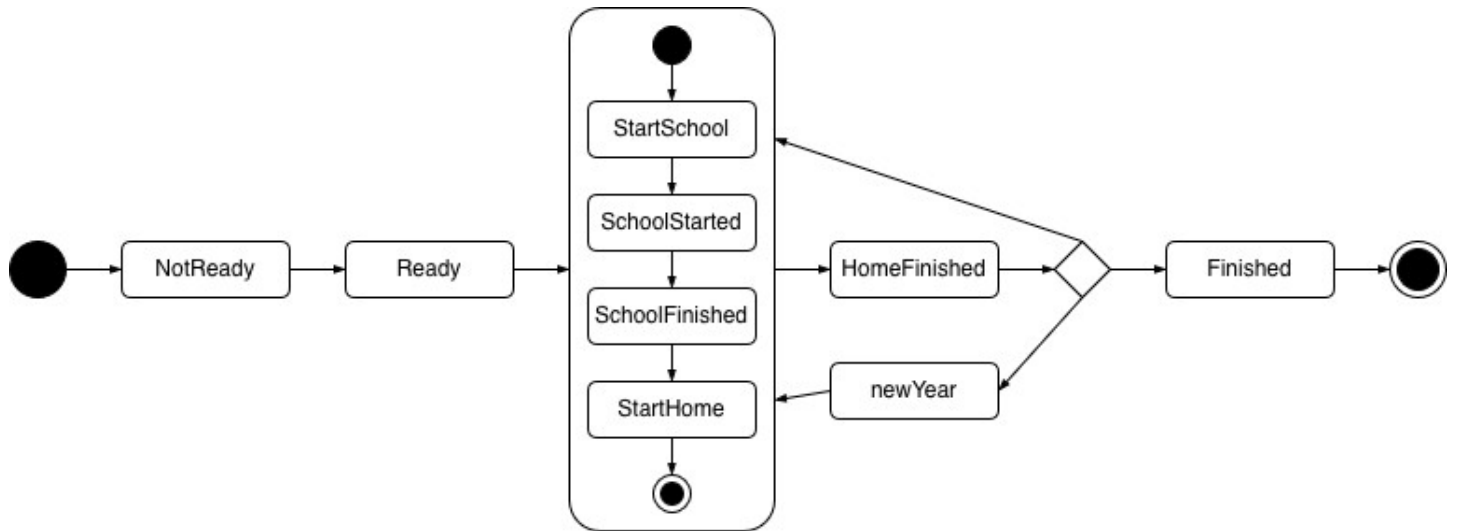
Synchroniser.java

Location: `src/env/Synchroniser.java`

Workspace: `sync`

The central coordination artefact that controls the simulation lifecycle through a state machine.

State machine



Static state

- `numFamily`, `numParents` – Set during initialisation.
- `homeGroups`, `numGroupsCreated` – Track home group creation during startup.
- `completedYearCount`, `cyclesCompleted` – Track simulation progress.
- `parentsFinished` – Counter for daily home-phase completion.
- `childrenFinalised` – Counter for end-of-simulation finalisation.
- `CYCLES_PER_YEAR = 365` – Days per simulated year.
- `ReentrantLock lock` – Prevents duplicate end-of-day/end-of-simulation processing.

Observable properties

Property	Type	Description
<code>status</code>	String	Current lifecycle state
<code>year</code>	int	Current year number (1-based)
<code>day</code>	int	Current day number (1-based)
<code>agent_age</code>	int	Current age of child agents
<code>children_finished</code>	int	Number of children that have finalised

Operations

`groupCreated(String groupName)` – Called by parent agents after creating a home group. When all families are registered, transitions to “Ready”.

`schoolReady()` – Called by the teacher. Transitions to “StartSchool”. Guarded: only accepts calls from `classroom_teacher`.

`startSchool()` – Called by the teacher. Transitions to “SchoolStarted”. Guarded: teacher only.

`finishSchool()` – Called by the teacher. Transitions through “SchoolFinished” to “StartHome”. Guarded: teacher only.

`finishedHome()` – Called by each parent agent. Increments `parentsFinished`. When the count reaches `numParents`, `tryLock()` ensures exactly one thread resets the counter and calls `endOfDay()`; concurrent threads that also see the condition met fail to acquire the lock and return without action.

`childAgentFinalised()` – Called by each child at simulation end. Increments `childrenFinalised` and updates the `children_finished` observable property, then blocks (via `await("allChildrenFinished")`) until all children have reported. Once released, `tryLock()` ensures only one thread writes output files via `DataLogger.writeFiles()`, shuts down the MAS via `RuntimeServicesFactory.get().stopMAS()`, and prints the output file locations.

Private methods

`endOfDay()` – Increments the day counter. If more days remain in the year, transitions to “StartSchool”. If the year is complete, signals `newYear`, increments the age, and either starts a new year or signals `finalise`.

`isTeacher(String agentName)` – Guard helper that checks if the calling agent is `classroom_teacher`.

DataLogger.java

Location: `src/env/DataLogger.java`

Workspace: `sync`

Records simulation statistics and writes output CSV files.

Static state

- `timestampString` – Formatted start time used in output filenames (YY-mm-dd-HH-MM-SS).
- `ArrayList<WordCountRow> wordCountRows` – Accumulated annual statistics.

Operations

`addAnnualStats(String agentName, String ses, int age, int uniqueSeen, int uniqueHeard, int learned)` – Called by child agents at each year-end. Appends a `WordCountRow` to the list.

Static methods

`initialise()` – Captures the start timestamp.

`writeFiles()` – Called by the Synchroniser at simulation end. Writes two CSV files:

1. **Summary file** (YY-mm-dd-HH-MM-SS `summary.csv`): Columns: Name, SES, Age, Unique_Seen, Unique_Heard, Words_Learned.
2. **Word list file** (YY-mm-dd-HH-MM-SS `word_list.csv`): Columns: Word, Name, SES, Age.

`outputSummaryLocation()` / `outputWordsLocation()` – Return the output file paths.

WordsLearnt.java

Location: `src/env/WordsLearnt.java`

Workspace: `sync`

Tracks which words each agent learned, used for the word list CSV output.

Data structure

`HashMap<String, ArrayList<AgentDetails>> wordAoA` – Maps words to lists of `AgentDetails` records (name, SES, age, word). Uses a private record `AgentDetails`.

Operations

`addLearnedWords(String agentName, String ses, Object[] wordAoa)` – Called by child agents during finalisation. Parses the `AgentSpeak` term array (format: `aoa(word, Age)`), extracting word and age via string splitting on `[(), "]`. Deduplicates by checking if the agent already has an entry for each word.

Static methods

`getAll()` – Returns all entries as `ArrayList<WordLearntRow>`, used by `DataLogger.writeFiles()`.

Organisational Layer (Moise)

learning_environ.xml

Location: src/org/learning_environ.xml

Defines the organisational specification using Moise's XML schema.

Structural specification

Roles: adult, child, offspring, parent, student, teacher

Group hierarchy:

```
learning_environment (root)
+-- classroom
|   +-- teacher (min: 1, max: 1)
|   +-- student (min: 1, max: unbounded)
+-- village
    +-- adult (min: 1)
    +-- child (min: 1)
    +-- indiv_home (sub-group, min: 1)
        +-- parent (min: 1, max: 2)
        +-- offspring (min: 1, max: 1)
```

Links:

- Teacher has authority and bi-directional communication with students (intra-group).

Formation constraints:

- offspring and student roles are compatible (bi-directional), allowing a single agent to hold both roles simultaneously.

Functional specification

Scheme classroom_reading_books:

- Root goal: tell_class_to_read_a_book (sequential plan)
 - Sub-goal 1: find_my_students
 - Sub-goal 2: read_books
- Mission tellStudentsToRead: encompasses both sub-goals.

Scheme speak_to_child:

- Root goal: speak_an_utterance_to_child (sequential plan)
 - Sub-goal: speak_utterance
- Mission speakUtterances: encompasses the sub-goal.

Normative specification

Norm	Type	Role	Mission
parentNorm	obligation	parent	speakUtterances
teacherNorm	obligation	teacher	tellStudentsToRead

JaCaMo Project File

File: child_vocabulary_acquisition.jcm

Defines the static MAS configuration:

Statically-defined agent:

- `classroom_teacher` – Source: `teacher.asl`, focuses on `classroom.library` and `sync.synchroniser`, plays teacher role in class group.

Workspaces and artefacts:

- `sync`: Synchroniser, DataLogger, WordsLearnt
- `classroom`: Library
- `home_environ`: Utterances

Organisation:

- `learning_environ` with group `class` (type: `classroom`) responsible for scheme `readingBooksClassroom` (type: `classroom_reading_books`), and group `village`.

Platform configuration:

`platform: local(asych_shared, 64, 5, 5, 10)`

This configures Jason’s threading model:

- `asych_shared` – Asynchronous shared thread pool
- `64` – Number of threads in the pool
- `5, 5, 10` – Reasoning cycle parameters (sense, deliberate, act repetitions per scheduling slot)

The large thread pool prevents deadlocks during startup with many agents, even when the host CPU has fewer cores.

Simulation Lifecycle

Startup Sequence

1. `VocabLauncher.main()` initialises JaCaMo runtime services.
2. `create()` sets up platforms, environments, organisations, and agents.
3. `createAgs()` loads configuration, initialises artefacts, and creates dynamic agents.
4. `start()` launches all agents.
5. Parent agents create home groups and signal `groupCreated`.
6. Once all groups are created, the synchroniser transitions to “Ready”.
7. Teacher executes `find_my_students` and signals `schoolReady`.
8. Synchroniser transitions to “StartSchool”, beginning the first day.

Daily Cycle

1. **Synchroniser** status: `StartSchool`
2. **Teacher** signals `startSchool` (status becomes `SchoolStarted`), selects a book, and either reads aloud or instructs students.
3. **Children** process words (seen or heard), attempt to learn each one.
4. **Teacher** receives completion confirmations, signals `finishSchool`.
5. **Synchroniser** transitions through `SchoolFinished` to `StartHome`.
6. **Parents** select and deliver utterances to their children (20 batches each).
7. **Children** process utterances, confirm completion.
8. **Parents** signal `finishedHome`.
9. **Synchroniser** (once all parents have reported, via counter and `tryLock()`) calls `endOfDay()`.

Yearly Progression

After 365 daily cycles:

1. Synchroniser signals `newYear`.
2. Child agents report annual statistics to DataLogger.
3. Child agents update their age belief.

4. Synchroniser increments the year counter and resets the day counter.
5. If more years remain, the cycle repeats from StartSchool.
6. If all years are complete, the synchroniser signals finalise.

Shutdown Sequence

1. Child agents receive the finalise signal.
2. Each child reports its learned words to WordsLearnt and signals childAgentFinalised.
3. Once all children have reported (via guard allChildrenFinished):
 - `DataLogger.writeFiles()` writes both CSV files.
 - `RuntimeServicesFactory.get().stopMAS()` initiates JaCaMo shutdown.
 - Output file locations are printed to the console.

Word Learning Algorithm

Each child agent maintains per-word exposure counts. A word is considered “learnt” when any of the following conditions is met:

Condition	Threshold
Seen (read) only	> 20 times
Heard (speech) only	> 20 times
Both seen and heard	> 12 times each

The learning check is performed in `try_learn_word(Word)` in `child.asl`:

```
if ((Seen > 12 & Heard > 12) | Seen > 20 | Heard > 20) {
    // Mark word as learnt at current age
}
```

Once learnt, a word’s `AgeLearnt` field is set to the child’s current age and is never re-evaluated.

Concurrency and Synchronisation

The simulation involves significant concurrency challenges due to many agents operating simultaneously. Key mechanisms:

Atomic plans (`@atomic`): Used extensively in child and teacher agents to prevent interleaving of plan execution. Critical for maintaining belief-base consistency, especially during word processing and state transitions.

CARTAgO guards (`@GUARD`): The Synchroniser uses the `allChildrenFinished` guard to block the `childAgentFinalised` operation (via `await`) until all children have reported. Agents calling the guarded operation are suspended by `CARTAgO` until the guard evaluates to `true`.

ReentrantLock: Used in two contexts:

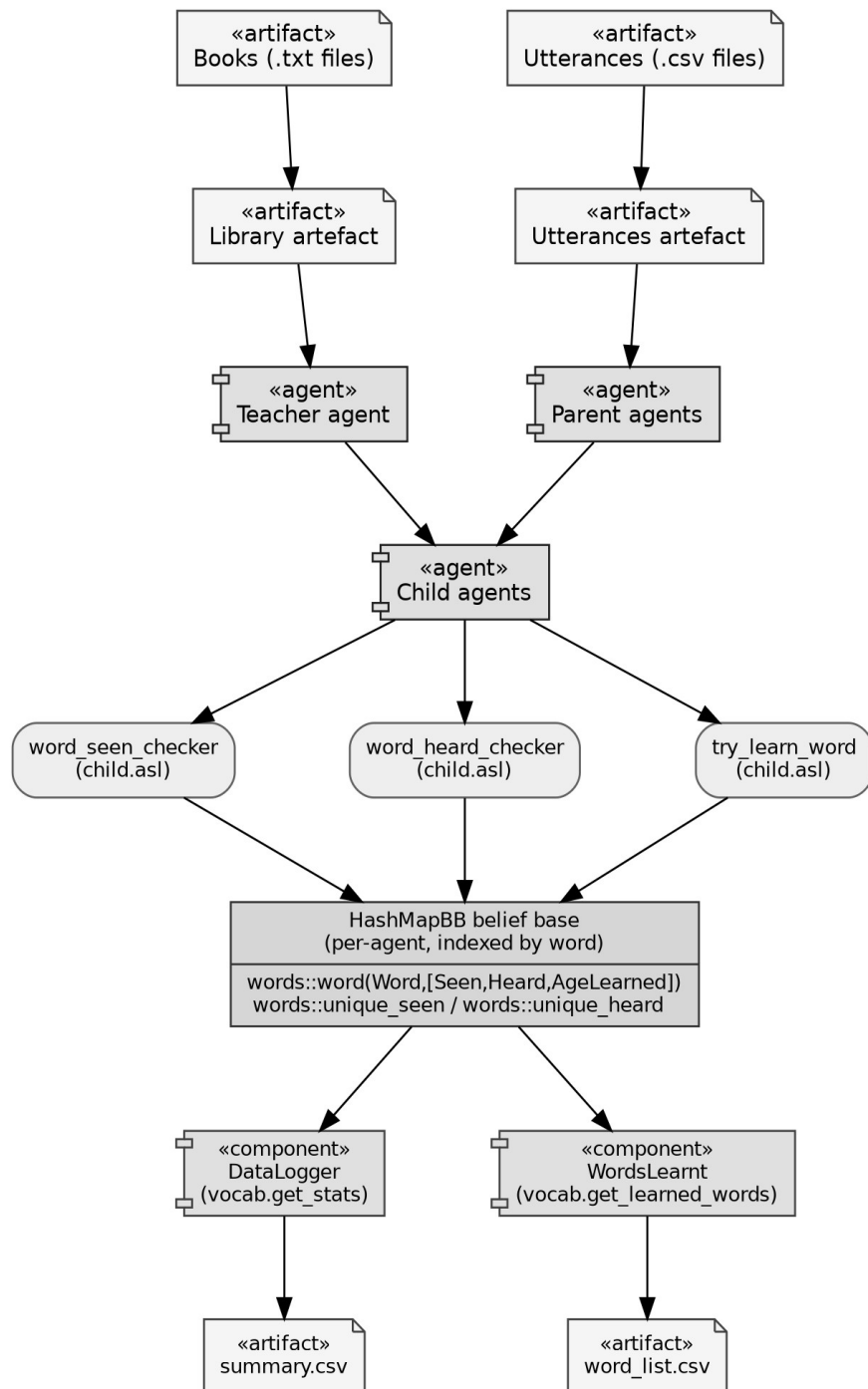
1. `Synchroniser.finishedHome()` – Each parent increments a counter; when all parents have reported, `tryLock()` ensures exactly one thread calls `endOfDay()`. Other threads see the counter is met but fail to acquire the lock and skip the call.
2. `Synchroniser.childAgentFinalised()` – After the `allChildrenFinished` guard releases all waiting agents, `tryLock()` ensures only one thread executes the shutdown sequence (writing output files and stopping the MAS).
3. `Library` – Ensures thread-safe lazy loading of book contents.

HashMapBB (custom belief base): A custom `BeliefBase` implementation (`src/env/HashMapBB.java`) that provides $O(1)$ indexed lookups for registered belief patterns. Each agent has its own `HashMapBB` instance. Indexed beliefs (e.g., `word(key, _)`) are stored in `HashMap<String, Literal>` keyed by a designated argument, while non-indexed beliefs use namespace-aware linked lists. The `getAsDOM()` method (used by the Mind Inspector) includes retry logic for `ConcurrentModificationException`.

Belief-base `.wait()` calls: Jason's `.wait(condition)` suspends the agent's intention until the specified belief condition becomes true. Used for inter-agent coordination (e.g., waiting for group formation, waiting for idle state).

Batched utterance delivery: Parent agents send utterances in 20 batches rather than one large message. The child uses a counter (`finished(Counter, Parent)`) to track received batches and only signals completion after all 20 are processed. This prevents overwhelming the agent's messaging system.

Data Flow



Logging Configuration

File: `logging.properties`

- Default log level: `WARN`
- Console handler: `java.util.logging.ConsoleHandler` with `jacamo.util.LogFormatter`
- File handler: Disabled by default; can be enabled by uncommenting the `FileHandler` configuration (outputs to `log/mas-*.log`).

Known Limitations and Unimplemented Features

1. **Siblings** – The `max_siblings` configuration is parsed but `addSiblingAgents()` is a no-op.
2. **Single teacher** – The system supports exactly one teacher agent. Multiple classrooms are not supported.
3. **Static word learning thresholds** – The learning criteria (`>20` or `>12+12`) are hard-coded in `child.asl`.
4. **Static fields in artefacts** – All artefacts use static fields, meaning only one instance of each artefact type can exist in the system.
5. **No incremental output** – Results are only written at simulation end. A crash mid-simulation loses all data.
6. **Memory usage** – Each child agent's word data is stored in its `HashMapBB` belief base (indexed by word), so large vocabularies with many agents can consume significant heap space.